

Reviewer Pack — Minimal Rank of Formal Language Automorphism Groups over Ran...

Entry 68196faaa036 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 15:24:47 UTC

Minimal Rank of Formal Language Automorphism Groups over Random SAT Satisfiability

Entry ID: 68196faaa036 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 15:24:47 UTC

1. Conjecture

Field A (mathematical branch): Automata Theory (Formal Language Automorphism Groups) **Field B** (complexity object): Complexity Theory: Random SAT Satisfiability

Statement:

[‘For a random 3-SAT formula with n variables, the minimal rank of its associated formal language automorphism group is $\Theta(\log^2 n)$.’, ‘Equivalently, for any two distinct clauses in the formula, there exists an automorphism that swaps their truth assignments.’]

Rationale (proposer’s reasoning):

[‘Formal language automorphism groups can capture structural symmetries in computational problems, potentially revealing hidden complexities.’, ‘The conjecture proposes a quantitative relation between automorphism group rank and the inherent complexity of random SAT formulas, suggesting a novel approach to understanding satisfiability problems.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1d454d63bfcdcf24

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

We consider a result supported if, across all 30 seeds, the computed minimal ranks of formal language automorphism groups for random 3-SAT formulas with n variables are within $\Theta(\log^2 n)$ of each other and Spearman’s rank correlation coefficient is ≥ 0.7 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal language automorphism groups AND minimal rank AND random SAT satisfiability - automata theory application IN random 3-SAT minimization of formal language automorphism group - truth assignment swapping automorphisms in random SAT clauses

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    min_rank_sum = 0

    for _ in range(instances_tested):
        clauses = []
        for _ in range(n // 3):
            clause = [random.choice([f'x{i}', f'~x{i}']) for i in range(1, n + 1)]
            random.shuffle(clause)
            clauses.append(' | '.join(clause))

        # Placeholder for actual computation of minimal rank
        min_rank = sum(len(set(clause.split(' | '))) for clause in clauses) / instances_tested

        min_rank_sum += min_rank

    metric_value = min_rank_sum / instances_tested
    conjecture_holds = math.log(n, 2)**2 * 0.8 <= metric_value <= math.log(n, 2)**2 * 1.2
    counterexample = "" if conjecture_holds else f"rank={metric_value}, expected=O(log^2 {n})"

    return {
        "metric_name": "minimal_rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **result}}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'seed': 11, **result}
TRIAL: {'seed': 23, **result}
TRIAL: {'seed': 37, **result}
TRIAL: {'seed': 53, **result}
TRIAL: {'seed': 71, **result}
TRIAL: {'seed': 89, **result}
TRIAL: {'seed': 103, **result}
TRIAL: {'seed': 127, **result}
TRIAL: {'seed': 149, **result}
TRIAL: {'seed': 167, **result}
TRIAL: {'seed': 191, **result}
TRIAL: {'seed': 211, **result}
TRIAL: {'seed': 233, **result}
TRIAL: {'seed': 257, **result}
TRIAL: {'seed': 277, **result}
TRIAL: {'seed': 311, **result}
TRIAL: {'seed': 347, **result}
TRIAL: {'seed': 389, **result}
TRIAL: {'seed': 421, **result}
TRIAL: {'seed': 463, **result}
TRIAL: {'seed': 503, **result}
TRIAL: {'seed': 547, **result}
TRIAL: {'seed': 593, **result}
TRIAL: {'seed': 631, **result}
TRIAL: {'seed': 677, **result}
TRIAL: {'seed': 727, **result}
TRIAL: {'seed': 773, **result}
TRIAL: {'seed': 821, **result}
TRIAL: {'seed': 877, **result}

```

```
TRIAL: {'seed': 929, **result}
```

```
--STDERR--
```

```
Traceback (most recent call last):
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7c76842.py", line 64, in <module>
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7c76842.py", line 64, in <genexpr>
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
                        ~~~~~^
```

```
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12743
2	propose	ollama_remote	glm4:latest	0	0	11014
3	preregistration	ollama_remote	glm4:latest	0	0	5972
4	novelty	ollama_remote	glm4:latest	0	0	4656
5	novelty	ollama_remote	glm4:latest	0	0	5810
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11657
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9137
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10260
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6763
10	judge	ollama_remote	glm4:latest	0	0	8902

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86913 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/68196faaa036.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/68196faaa036.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/68196faaa036.tar.gz` (if generated)